

CSCI3170

Introduction to Database Systems

5-sql_intro
Slides: Michael Yu

Introduction

- **Structured Query Language (SQL)** most widely used commercial relational database language
- SQL language has several aspects:
 - **Data Manipulation Language (DML)**: Allows users to pose queries and to insert, delete, and modify rows.
 - **Data Definition Language (DDL)**: Supports the creation, deletion, and modification to definitions for tables and views.
 - **Triggers and Advanced Integrity Constraints**: Action executed by the DBMS whenever changes to database meet conditions specified in trigger

Basic Query Structure

- To retrieve information from a database, there is a basic query structure, known as the ***select*** statement.
 - **SELECT** <attribute list>
FROM <table list>
WHERE <condition>
- SELECT statement is also known as a ***select-from-where block***.

SELECT Clause

- lists the attributes desired in the result of a query
 - corresponds to the **projection** operation of the relational algebra
- Query: Return names of all drinkers

– **SELECT** Name
FROM Drinkers;

Drinkers:

Name	Addr	Phone
Adam	Randwick	9385-4444
Gernot	Newtown	9415-3378
John	Clovelly	9665-1234
Justin	Mosman	9845-4321

- *Note: FROM is always necessary with SELECT, whereas WHERE is optional.*

SELECT Clause - Examples

- Query: Return names and addresses of drinkers

– **SELECT** **Name**, **Addr**
FROM Drinkers;

Drinkers:

Name	Addr	Phone
Adam	Randwick	9385-4444
Gernot	Newtown	9415-3378
John	Clovelly	9665-1234
Justin	Mosman	9845-4321

- An **asterisk (*)** in denotes “all attributes”

– **SELECT** *****
FROM Drinkers;

Drinkers:

Name	Addr	Phone
Adam	Randwick	9385-4444
Gernot	Newtown	9415-3378
John	Clovelly	9665-1234
Justin	Mosman	9845-4321

SELECT Clause - DISTINCT

- To eliminate duplicates in the query results, insert the keyword ***distinct*** after select.
- Query: Return unique department names

– **SELECT** ***DISTINCT*** dept_name
FROM instructor

- Note: keyword **all** specifies that duplicates not to be removed

– **SELECT** ***ALL*** dept_name
FROM instructor

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

A Note on DISTINCT

- SQL is based on multisets, RA is originally based on sets
 - Set: {a, b, c}
 - Multiset: {a, a, b, b, c, a, a, b, c, c ...}
- On the practical handling of distinct and duplicate tuples
 - An SQL table can either simple set of tuples, or multiset of tuples
 - Aka. SQL allows duplicates in relations and in query results
 - allows table to have two or more tuples that are identical in all their attribute values

FROM Clause

- The from clause lists the relations involved in the query
 - Corresponds to the ***Cartesian product*** operation of the relational algebra.
- Find the Cartesian product *instructor* $\times$ *teaches*
 - **SELECT ***
FROM *instructor, teaches*
 - generates every possible instructor – teaches pair, with all attributes from both relations.
- *Cartesian product not very useful directly, but useful combined with where-clause condition (selection operation in relational algebra).*

FROM Clause

instructor

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000

teaches

ID	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009

SELECT * FROM *instructor, teaches*

<i>Inst.ID</i>	name	dept_name	salary	<i>teaches.ID</i>	course_id	sec_id	semester	year
10101	Srinivasan	Comp. Sci	65000	10101	CS-101	1	Fall	2009
10101	Srinivasan	Comp. Sci	65000	10101	CS-315	1	Spring	2010
10101	Srinivasan	Comp. Sci	65000	10101	CS-347	1	Fall	2009
10101	Srinivasan	Comp. Sci	65000	12121	FIN-201	1	Spring	2010
10101	Srinivasan	Comp. Sci	65000	15151	MU-199	1	Spring	2010
10101	Srinivasan	Comp. Sci	65000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
12121	Wu	Finance	90000	10101	CS-101	1	Fall	2009
12121	Wu	Finance	90000	10101	CS-315	1	Spring	2010
12121	Wu	Finance	90000	10101	CS-347	1	Fall	2009
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2010
12121	Wu	Finance	90000	15151	MU-199	1	Spring	2010
12121	Wu	Finance	90000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...

WHERE Clause

- **WHERE** clause specifies conditions that the result must satisfy
- Query: Return bars that sells the beer *New*

```
– SELECT *  
  FROM Sells  
  WHERE Beer = 'New';
```

```
– SELECT Name  
  FROM Beers  
  WHERE Manf = 'Toohey''s';
```

Sells:

Bar	Beer	Price
Australia Hotel	Burraborang Bock	3.5
Regent Hotel	New	2.2
Regent Hotel	Victoria Bitter	2.2

If string has single quotes? Escape by doubling them (‘’) informs SQL that the quote is part of the string literal.

Without escaping is error inducing (see below)

SELECT Name FROM Beers WHERE Manf = ‘Toohey’s’;

On "Joining" Tables

- Query: Return sids of sailors who have reserved a red boat

```
– SELECT  R.sid  
  FROM   Boat B, Reserves R  
  WHERE   B.bid = R.bid AND B.color = 'red';
```

- Query: Return names of sailors who have reserved a red boat

```
– SELECT  S.sname  
  FROM   Sailors S, Reserves R, Boat B  
  WHERE   S.sid = R.sid AND R.bid = B.bid AND  
           B.color = 'red';
```

sailors	<table><tr><td><u>sid</u></td><td>sname</td><td>rating</td><td>age</td></tr></table>	<u>sid</u>	sname	rating	age
<u>sid</u>	sname	rating	age		
Boats	<table><tr><td><u>bid</u></td><td>bname</td><td>color</td></tr></table>	<u>bid</u>	bname	color	
<u>bid</u>	bname	color			
Reserves	<table><tr><td><u>sid</u></td><td><u>bid</u></td><td><u>day</u></td></tr></table>	<u>sid</u>	<u>bid</u>	<u>day</u>	
<u>sid</u>	<u>bid</u>	<u>day</u>			

On "Joining" Tables

- Query: Return the colors of boats reserved by Lubber

```
- SELECT  B.color
  FROM    Sailors S, Reserves R, Boat B
 WHERE    S.sid = R.sid AND R.bid = B.bid AND
          S.sname = 'Lubber';
```

- (In general, there may be more than one sailor called Lubber. In this case, it will return the colors of boats reserved by all sailors called Lubber).

sailors	<table><tr><td><u>sid</u></td><td>sname</td><td>rating</td><td>age</td></tr></table>	<u>sid</u>	sname	rating	age
<u>sid</u>	sname	rating	age		
Boats	<table><tr><td><u>bid</u></td><td>bname</td><td>color</td></tr></table>	<u>bid</u>	bname	color	
<u>bid</u>	bname	color			
Reserves	<table><tr><td><u>sid</u></td><td><u>bid</u></td><td><u>day</u></td></tr></table>	<u>sid</u>	<u>bid</u>	<u>day</u>	
<u>sid</u>	<u>bid</u>	<u>day</u>			

On "Joining" Tables

- Query: Find names of sailors who have reserved at least one boat

```
– SELECT  S.sname  
  FROM    Sailors S, Reserves R  
  WHERE   S.sid = R.sid ;
```

- (If a sailor has not made a reservation, the second step in the conceptual evaluation strategy would eliminate all rows in the cross-product that involve this sailor).

sailors	<table><tr><td><u>sid</u></td><td>sname</td><td>rating</td><td>age</td></tr></table>	<u>sid</u>	sname	rating	age
<u>sid</u>	sname	rating	age		
Boats	<table><tr><td><u>bid</u></td><td>bname</td><td>color</td></tr></table>	<u>bid</u>	bname	color	
<u>bid</u>	bname	color			
Reserves	<table><tr><td><u>sid</u></td><td><u>bid</u></td><td><u>day</u></td></tr></table>	<u>sid</u>	<u>bid</u>	<u>day</u>	
<u>sid</u>	<u>bid</u>	<u>day</u>			

On "Joining" Tables

- Query: Of all instructors who had taught a course, return the instructor names and the ID of courses they taught
 - **SELECT** *name, course_id*
FROM *instructor, teaches*
WHERE *instructor.ID = teaches.ID ;*

instructor

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000

teaches

<i>ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009

On "Joining" Tables

- Query: Return IDs of courses taught in 2010, and the name of the instructor who taught them

```
– SELECT name, course_id  
  FROM instructor, teaches  
  WHERE instructor.ID = teaches.ID AND year =  
    2010 ;
```

instructor

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000

teaches

ID	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009

Summary on Semantics

- For SQL *SELECT* statement on several relations:

```
SELECT DISTINCT  $a_1, a_2, \dots, a_n$   
FROM  $R_1, R_2, \dots, R_m$   
WHERE P
```

An SQL query intuitively corresponds to a relational algebra expression involving selections, projections, and cross-products.

$$\Pi_{a_1, a_2, \dots, a_n} (\sigma_P (R_1 \times R_2 \times \dots \times R_m))$$

NATURAL JOIN

- **Natural join:** (1) matches tuples with same values for all common attributes, (2) **retains only one copy of each common column**

<u>ID</u>	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
76766	Crick	Biology	72000

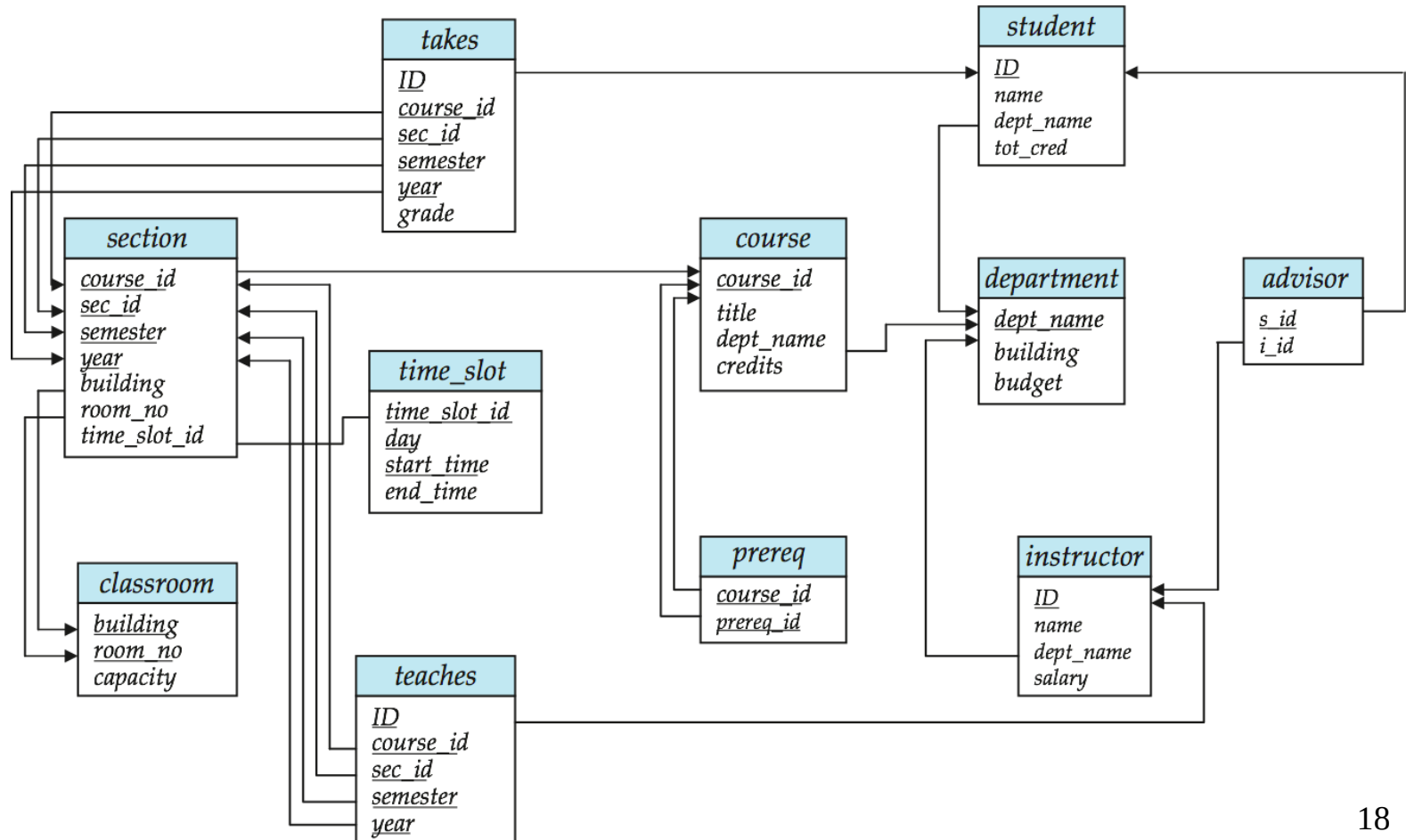
<u>ID</u>	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009

– **SELECT ***
FROM instructor NATURAL JOIN teaches;

<u>ID</u>	name	dept_name	salary	course_id	sec_id	semester	year
10101	Srinivasan	Comp. Sci.	65000	CS-101	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000	CS-315	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	CS-347	1	Fall	2009
12121	Wu	Finance	90000	FIN-201	1	Spring	2010
15151	Mozart	Music	40000	MU-199	1	Spring	2010
22222	Einstein	Physics	95000	PHY-101	1	Fall	2009
32343	El Said	History	60000	HIS-351	1	Spring	2010
45565	Katz	Comp. Sci.	75000	CS-101	1	Spring	2010
45565	Katz	Comp. Sci.	75000	CS-319	1	Spring	2010
76766	Crick	Biology	72000	BIO-101	1	Summer	2009
76766	Crick	Biology	72000	BIO-301	1	Summer	2010

DB Schema - University Database

- **SELECT** name, title
FROM teaches **NATURAL JOIN** instructor **NATURAL JOIN** course;



NATURAL JOIN

- Query: Returning names of instructors + titles of courses they teach

instructor

ID	name	dept_name	salary
8	ABC	SEEM	100
7	XYZ	SEEM	120

teaches

ID	course_id	sec_id	semester	year
7	3550	1	1	2018
8	2100	1	2	2018

course

course_id	title	dept_name	credits
3550	DB	SEEM	3
2100	Algo	CSE	3

Thoughts on this SQL command?

```
SELECT name, title
FROM teaches
NATURAL JOIN instructor
NATURAL JOIN course;
```

NATURAL JOIN - Caution

- Due to defined behavior of natural joins, be aware of unrelated attributes with same names – will get equated incorrectly
- Previous Query: Returning names of instructors + titles of courses they teach
 - Correction:
 - `select name, title`
`from instructor natural join teaches,`
`course`
`where teaches.course_id =`
`course.course_id;`

Ordering Result Tuples (Default: ASC)

- List in alphabetic order the names of all instructors
 - **SELECT DISTINCT** *name*
FROM *instructor*
ORDER BY *name*
- We may specify **DESC** for descending order or **ASC** for ascending order. e.g.,

- **ORDER BY** *name* **DESC**
- **ORDER BY** *name* **ASC**

- Can sort on multiple attributes: e.g.,

- **ORDER BY** *dept_name, name*

Divide Operation in SQL?

- Find bars each of which sell all beers Justin likes.
- Relational Algebra: $\pi_{\text{bar}, \text{beerSells}} / (\pi_{\text{beer}}(\sigma_{\text{drinker}='Justin'} \text{ Likes}))$

Sells:

Bar	Beer	Price
Australia Hotel	Burraborang Bock	3.5
Coogee Bay Hotel	New	2.25
Coogee Bay Hotel	Old	2.5
Coogee Bay Hotel	Sparkling Ale	2.8
Coogee Bay Hotel	Victoria Bitter	2.3
Lord Nelson	Three Sheets	3.75
Lord Nelson	Old Admiral	3.75
Marble Bar	New	2.8
Marble Bar	Old	2.8
Marble Bar	Victoria Bitter	2.8
Regent Hotel	New	2.2
Regent Hotel	Victoria Bitter	2.2
Royal Hotel	New	2.3
Royal Hotel	Old	2.3
Royal Hotel	Victoria Bitter	2.3

Likes:

Drinker	Beer
Adam	Crown Lager
Adam	Fosters Lager
Adam	New
Gernot	Premium Lager
Gernot	Sparkling Ale
John	80/-
John	Bigfoot Barley Wine
John	Pale Ale
John	Three Sheets
Justin	Sparkling Ale
Justin	Victoria Bitter

Divide Operation

- Find bars each of which sell all beers Justin likes:
 - $\pi_{\text{bar}, \text{beerSells}} / (\pi_{\text{beer}}(\sigma_{\text{drinker}='Justin'} \text{ Likes}))$
- Textbook way to translate division:
 - **Select** distinct a.bar
from sells a
where not exists
 ((select b.beer from likes b
 where b.drinker = 'Justin')
 except
 (select c.beer from sells c
 where c.bar = a.bar)
);

Set Operations

Note: Each of the below operations will eliminate duplicates. To keep duplicates, use **union all**, **intersect all**, **except all**.

- Find courses that ran in Fall 2009 or in Spring 2010

- `(Select course_id from section where sem = 'Fall' and year = 2009) union (Select course_id from section where sem = 'Spring' and year = 2010);`

- Find courses that ran in Fall 2009 and in Spring 2010

- `(Select course_id from section where sem = 'Fall' and year = 2009) intersect (Select course_id from section where sem = 'Spring' and year = 2010);`

- Find courses that ran in Fall 2009 but not in Spring 2010

- `(select course_id from section where sem = 'Fall' and year = 2009) except (select course_id from section where sem = 'Spring' and year = 2010);`

Find the brewers whose
beers John likes

```
SELECT Manf
FROM Likes, Beers
WHERE drinker = 'John' AND beer = name;
```

Likes:

Drinker	Beer
Adam	Crown Lager
Adam	Fosters Lager
Adam	New
Gernot	Premium Lager
Gernot	Sparkling Ale
John	80/-
John	Bigfoot Barley Wine
John	Pale Ale
John	Three Sheets
Justin	Sparkling Ale
Justin	Victoria Bitter

Beers:

Name	Manf
80/-	Caledonian
Bigfoot Barley Wine	Sierra Nevada
Burraborang Bock	George IV Inn
Crown Lager	Carlton
Fosters Lager	Carlton
Invalid Stout	Carlton
Melbourne Bitter	Carlton
New	Toohey's
Old	Toohey's
Old Admiral	Lord Nelson
Pale Ale	Sierra Nevada
Premium Lager	Cascade
Red	Toohey's
Sheaf Stout	Toohey's
Sparkling Ale	Cooper's
Stout	Cooper's
Three Sheets	Lord Nelson
Victoria Bitter	Carlton

Recall: Querying Multi-relations

- Example: Find brewers whose beers John likes

- **Schema**

- Likes(drinker, beer)
 - Beers(name, manf)

MANF

Caledonian
Sierra Nevada
Sierra Nevada
Lord Nelson

- **SELECT** Manf
FROM Likes, Beers
WHERE drinker = 'John' AND beer = name;
- Note: eliminate duplicates with ***DISTINCT***
- Relational algebra: $\pi_{manf}(\sigma_{drinker='John'} Likes \bowtie_{(beer, name)} Beers).$

Name Clashes - Attributes

- When attributes from diff relations share the same name
 - Same names can cause confusion in a WHERE clause
- Example: Which hotels has the same name as a beer?

- **SELECT** Bars.name
FROM Bars, Beers
WHERE name = name;

Schema

Beers (name, manf)

Bars (name, addr, license)

- Solution: Disambiguate attributes with their relation name
 - **SELECT** Bars.name
FROM Bars, Beers
WHERE Bars.name = Beers.name;

Name Clashes - Attributes

- You can qualified names even if there is no ambiguity:

```
– SELECT Sells.beer  
  FROM Sells  
  WHERE Sells.price > 3.00;
```

- Advice is to use qualify attribute names only if necessary

Name Clashes - Table

- Relation-dot-attribute convention doesn't help if we use same relation twice in SELECT
- To handle this, we need to define new names for each “instance” of the relation in FROM clause.

Name Clashes - Table

- Query: Return pairs of beers by same manufacturer
 - We should avoid pairing a beer with itself (*New,New*) and same pairs with different order (*New,Old*) (*Old,New*)
- SELECT** b1.name, b2.name
FROM Beers b1, Beers b2
WHERE
b1.manf = b2.manf AND
b1.name < b2.name;

NAME	NAME
-----	-----
Crown Lager	Fosters Lager
Crown Lager	Invalid Stout
Fosters Lager	Invalid Stout
Fosters Lager	Melbourne Bitter
....	

Beers:

Name	Manf
80/-	Caledonian
Bigfoot Barley Wine	Sierra Nevada
Burraborang Bock	George IV Inn
Crown Lager	Carlton
Fosters Lager	Carlton
Invalid Stout	Carlton
Melbourne Bitter	Carlton
New	Toohey's
Old	Toohey's
Old Admiral	Lord Nelson
Pale Ale	Sierra Nevada
Premium Lager	Cascade
Red	Toohey's
Sheaf Stout	Toohey's
Sparkling Ale	Cooper's
Stout	Cooper's
Three Sheets	Lord Nelson
Victoria Bitter	Carlton

Note on Range Variables

- Really needed only if same relation appears twice in the FROM clause
- Following queries are equivalent

- **SELECT** S.sname
FROM Sailors S, Reserves R
WHERE S.sid=R.sid AND bid=103;

- **SELECT** S.sname
FROM Sailors **AS** S, Reserves **AS** R
WHERE S.sid=R.sid AND bid=103;

- (Use of AS keyword is somewhat optional)

- **SELECT** sname
FROM Sailors, Reserves
WHERE Sailors.sid=Reserves.sid AND bid=103;

Other Usages of *via AS* (cont.)

- Example: Changing name of relation *Beers(name, manf)*
 - **SELECT** name **AS** Brand, manf **AS** Brewer
FROM Beers;

BRAND	BREWER
-----	-----
80/-	Caledonian
Bigfoot Barley Wine	Sierra Nevada
Burraborang Bock	George IV Inn
Crown Lager	Carlton
Fosters Lager	Carlton
Invalid Stout	Carlton
...	

Expressions as Values in Columns

- **AS** can introduce an attribute of computed values
- Example: *Sells(bar, beer, price)*

– **SELECT** bar, beer, price*120 **AS** PriceInYen
FROM Sells;

BAR	BEER	PRICEINYEN
-----	-----	-----
Australia Hotel	Burraborang Bock	420
Coogee Bay Hotel	New	270
Coogee Bay Hotel	Old	300
Coogee Bay Hotel	Sparkling Ale	336
Coogee Bay Hotel	Victoria Bitter	276
...		

Inserting Text in Result Table

- AS can be used to intro. an attribute that is a **constant expression**.
- Schema: **Likes(drinker, beer)**
- SELECT** drinker, 'likes Cooper's' AS WhoLikes
FROM Likes
WHERE beer = 'Sparkling Ale';

DRINKER	WHOLIKES
-----	-----
Gernot	likes Cooper's
Justin	likes Cooper's

Likes

Drinker	Beer
Adam	Crown Lager
Adam	Fosters Lager
Adam	New
Gernot	Premium Lager
Gernot	Sparkling Ale
John	80/-
John	Bigfoot Barley Wine
John	Pale Ale
John	Three Sheets
Justin	Sparkling Ale
Justin	Victoria Bitter

Expressions and Strings

- **SELECT** S.age, age1=S.age-5, 2*S.age AS age2
FROM Sailors S
WHERE S.sname **LIKE** 'B_%B'
- Illustrates use of arithmetic expressions and string pattern matching:
Find triples (of ages of sailors and two fields defined by expressions) for sailors whose names begin and end with B and contain at least three characters.
 - **AS** and **=** are two ways to name fields in result.
 - **LIKE** is used for string matching. **'_'** stands for any one character and **'%'** stands for 0 or more arbitrary characters.

Grouping

- SELECT-FROM-WHERE can be followed by GROUP BY to:
 - partition result relation into groups (according to values of specified attribute)
 - treat each group separately in computing aggregations
- Example: How many beers does each brewer make?

```
– SELECT manf, COUNT(beer)
  FROM Beers
  GROUP BY manf;
```

MANF	COUNT(beer)
-----	-----
Caledonian	1
Carlton	5
Cascade	1
Cooper's	2
George IV Inn	1
Lord Nelson	2
Sierra Nevada	2
Toohey's	4

Grouping(cont.)

- GROUP BY is used as follows:
 - **SELECT** attributes/**aggregations**
FROM relations
WHERE condition
GROUP BY attribute
- Semantics:
 - partitions result into groups based on distinct values of attribute
 - apply any aggregation separately to each group

Grouping(cont.)

- Grouping is typically used in queries involving the phrase “for each”.
- Query: For each drinker, find average price of *New* at bars they frequently go to.

```
- SELECT drinker, AVG(price)
  FROM Frequents, Sells
 WHERE beer = 'New' AND Frequents.bar =
      Sells.bar
 GROUP BY drinker;
```

DRINKER	AVG(PRICE)
-----	-----
Adam	2.25
John	2.25
Justin	2.5

Schema:

Sells(bar, beer, price)
Frequents(drinker, bar)

Grouping(cont.)

- Query: Find the cheapest beer price in each bar.

- **SELECT** bar, MIN(price)
FROM Sells
GROUP BY BAR

bar	MIN(PRICE)
-----	-----
Australia Hotel	3.5
Coogee Bay Hotel	2.25
Lord Nelson	3.75
Marble Bar	2.8
Regent Hotel	2.2
Royal Hotel	2.3

- Incorrect usage:

- **SELECT** bar, MIN(price)
FROM Sells;

- *ERROR: column "sells.bar" must appear in the GROUP BY clause or be used in an aggregate function LINE 1: select bar, min(price) from sells;*

- When using grouping, every attribute in the SELECT list must: have an aggregation operator applied to it OR appear in a GROUP-BY clause

Eliminating Groups

- Example: Average beer price by suburb excluding hotels in The Rocks.

```
- SELECT Bars.addr, AVG(Sells.price)
  FROM Sells, Bars
  WHERE Bars.addr != 'The Rocks' AND Sells.bar
    = Bars.name
  GROUP BY Bars.addr;
```

ADDR	AVG(SELLS.PRICE)
-----	-----
Coogee	2.4625
Kingsford	2.2
Randwick	2.3
Sydney	2.8

Eliminating Groups

- You can impose complex conditions on groups, use the **HAVING** clause. HAVING is used to qualify a GROUP-BY clause:
 - **SELECT** attributes/aggregations
FROM relations
WHERE condition
GROUP BY attribute
HAVING condition;
- Operational Semantics:
 1. generate groups as for GROUP-BY
 2. eliminate any group not satisfying HAVING condition
 3. apply an aggregation to remaining groups

Eliminating Groups(cont.)

- Query: Find average price of popular beers (i.e. those that are served in more than one hotel).

```
– SELECT beer, AVG(price)
   FROM Sells
   GROUP BY beer
   HAVING COUNT(bar) > 1;
```

BEER	AVG(PRICE)
-----	-----
New	2.3875
Old	2.53333333
Victoria Bitter	2.4